

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática



Computação Gráfica

2025/2026

Fase 3

VBOs, Animações e *Bezier Patches*

GRUPO 22

106804, Alice Isabel Faria Soares

106853, Ana Beatriz Freitas

107365, Beatriz Martins Miranda

107367, João Paulo Batista Azevedo

Maio 2026

Índice

1	Introdução	2
2	VBOs	3
2.1	Formato dos modelos	3
2.2	Uso de índices	3
2.3	Upload para a GPU	3
2.4	Renderização via VBOs	3
3	Transformações Temporais	4
3.1	Medição do tempo	4
3.2	Rotações animadas	4
3.3	Translações animadas - Curva de <i>Catmull-Rom</i>	4
3.3.1	Alinhamento à curva	5
3.3.2	Visualização da curva	5
4	Bezier Patches	6
4.1	Formato do ficheiro .patch	6
4.2	Geração	6
4.3	Normais	6
5	Testes	7
6	Sistema Solar Dinâmico	8
6.1	Planetas e luas	8
6.2	Cometa	8
7	Conclusão	9

1 Introdução

O presente relatório descreve o trabalho realizado na terceira fase do trabalho prático da unidade curricular de Computação Gráfica, inserida no 2º Semestre do 3º Ano da Licenciatura em Engenharia Informática, na Universidade do Minho.

Esta fase assenta em três grandes objetivos, nomeadamente, em substituir o modo imediato de renderização por VBOs, *Vertex Buffer Objects*, eliminando a transferência repetida de dados entre CPU e GPU em cada *frame*. Passa ainda pela introdução de transformações temporais no *engine*, nomeadamente rotações contínuas e translações sobre curvas de *Catmull-Rom*, com suporte a alinhamento automático do objeto à tangente da curva. Por fim estende o *generator* com suporte à criação de modelos a partir de *Bezier Patches*, permitindo representar superfícies curvas de forma paramétrica.

Como *scene* de demonstração, o sistema solar estático da fase anterior foi transformado num sistema dinâmico e animado, com todos os planetas e luas a orbitar com períodos próprios, acrescido de um cometa cuja trajetória é definida por uma curva de *Catmull-Rom* e cujo modelo é gerado a partir de *patches* de *Bézier*.

2 VBOs

Nas fases anteriores, a renderização dos modelos era feita em modo imediato. Deste modo, em cada *frame*, os vértices eram enviados um a um da CPU para a GPU. Esta abordagem implica comunicação contínua entre CPU e GPU, tornando-se ineficiente à medida que a complexidade da *scene* aumenta.

A solução implementada consiste em carregar toda a geometria para a GPU uma única vez no arranque da aplicação, guardando os dados em *buffers* de memória na GPU, *Vertex Buffer Objects*. A partir deste ponto, cada *frame* apenas referencia os *buffers* já existentes na GPU, sem qualquer transferência adicional.

2.1 Formato dos modelos

O formato do ficheiro `.3d` manteve-se inalterado face às fases anteriores. Deste modo, a primeira linha contém o número total de vértices, e cada linha seguinte representa um vértice com oito valores, sendo estes, posição (x, y, z) , normal (n_x, n_y, n_z) e coordenadas de textura (u, v) .

Estes oito valores estão mapeados diretamente para uma estrutura *Vertex* de 32 bytes, armazenada de forma *interleaved*, o que permite o *upload* direto para a GPU sem necessidade de reformatação dos dados.

2.2 Uso de índices

Para cada modelo, o *engine* percorre os vértices carregados e constrói uma lista de vértices únicos. Esta deduplicação é feita associando cada vértice ao índice onde foi guardado.

Quando um vértice já existe, é reutilizado o índice anterior. Quando aparece pela primeira vez, é adicionado à lista de vértices únicos e é criado um novo índice. Assim, cada modelo passa a ter:

- um VBO com os vértices únicos;
- um *index buffer* com a ordem dos vértices dos triângulos.

2.3 Upload para a GPU

Após o carregamento dos modelos a partir do XML e a criação do contexto OpenGL, é executada uma travessia recursiva da árvore de grupos da *scene*.

Para cada modelo, é gerado um identificador de *buffer*, os dados são enviados para a GPU com a *flag* `GL_STATIC_DRAW`, indicando que a geometria não se altera durante a execução, e o identificador e a contagem de vértices são guardados para uso na renderização.

2.4 Renderização via VBOs

Na renderização, o VBO de vértices de cada modelo é ativado e é indicado à GPU o *layout* dos dados, isto é, são usados três componentes float para a posição, começando no início da estrutura *Vertex*, e o stride é `sizeof(Vertex)`, uma vez que cada vértice contém também normal e coordenadas de textura. A geometria é desenhada com uma única chamada à GPU por malha, sem iteração em CPU sobre os vértices durante a renderização.

3 Transformações Temporais

Nesta fase, os elementos `translate` e `rotate` do XML foram estendidos para suportar um atributo `time`. Quando este atributo existe, a transformação deixa de ser estática e passa a ser dinâmica, dependendo do tempo decorrido desde o início da execução. As transformações estáticas das fases anteriores continuam a ser suportadas em simultâneo.

3.1 Medição do tempo

Para suportar animações baseadas em tempo, é necessário conhecer, em cada *frame*, o instante atual da simulação. O tempo é obtido através da função GLUT, `glutGet(GLUT_ELAPSED_TIME)`, que devolve o número de milissegundos decorridos desde o início do programa. Esse valor é convertido para segundos e guardado numa variável global t antes de cada renderização:

$$t = \frac{\text{glutGet}(\text{GLUT_ELAPSED_TIME})}{1000}$$

Ao usar o tempo absoluto desde o arranque, em vez de somar incrementos Δt entre *frames*, evita-se a acumulação de erros de vírgula flutuante ao longo de execuções prolongadas. O valor t é depois usado diretamente em todas as transformações temporais durante a travessia da árvore de grupos, nomeadamente, nas rotações animadas para calcular o ângulo atual, e nas translações de *Catmull-Rom* para determinar a posição e orientação ao longo da curva.

3.2 Rotações animadas

Uma rotação temporal recebe como parâmetros o eixo de rotação e a duração de uma rotação completa em segundos. O ângulo de rotação aplicado no instante t é calculado pela fórmula:

$$\alpha(t) = \left(\frac{t}{\text{time}} \times 360 \right) \bmod 360$$

O ângulo é mantido no intervalo $[0^\circ, 360^\circ]$ para evitar imprecisões numéricas ao longo do tempo. O resultado é passado diretamente à função de rotação do OpenGL com o eixo definido.

3.3 Translações animadas - Curva de *Catmull-Rom*

A translação temporal anima um grupo ao longo de uma curva de *Catmull-Rom* fechada, definida por um conjunto de pontos de controlo no XML, mínimo de 4. O tempo global normalizado $gt \in [0, n[$, onde n é o número de segmentos, é decomposto num índice de segmento seg e num parâmetro local t :

$$seg = \lfloor gt \rfloor \bmod n$$

$$t = gt - \lfloor gt \rfloor \in [0, 1[$$

Para cada segmento, os quatro pontos de controlo adjacentes P_0, P_1, P_2, P_3 determinam a posição e a derivada da curva através da multiplicação pela matriz característica de *Catmull-Rom*:

$$M_{CR} = \begin{pmatrix} -\frac{1}{2} & \frac{3}{2} & -\frac{3}{2} & \frac{1}{2} \\ 1 & -\frac{5}{2} & 2 & -\frac{1}{2} \\ -\frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Calculando $C = M_{CR} \cdot (P_0 \ P_1 \ P_2 \ P_3)^T$, a posição e a derivada no parâmetro local $t \in [0, 1[$ são então:

$$q(t) = (t^3 \ t^2 \ t \ 1) \cdot C$$

$$q'(t) = (3t^2 \ 2t \ 1 \ 0) \cdot C$$

A curva é fechada, isto é, o índice do segmento é calculado modularmente, pelo que os pontos de controlo envolvem os extremos da lista, garantindo continuidade na junção.

3.3.1 Alinhamento à curva

Quando o atributo `align` está ativo, o objeto é orientado de forma a que o seu eixo local fique tangente à curva em cada instante. A partir da derivada $q'(t)$, constrói-se um referencial ortonormal (X, Y, Z) :

$$\vec{X} = \text{normalize}(\vec{q}'(t))$$

$$\vec{Z} = \text{normalize}(\vec{X} \times \vec{Y}_{ant})$$

$$\vec{Y} = \text{normalize}(\vec{Z} \times \vec{X})$$

O vetor Y_{ant} é inicializado a $(0, 1, 0)$, ou $(0, 0, 1)$ caso X seja paralelo a $(0, 1, 0)$, sendo armazenado em cada `TransformOp`. Durante o cálculo do alinhamento, este vetor é atualizado e reutilizado nos *frames* seguintes, permitindo propagar a orientação ao longo da curva. Como cada transformação animada possui o seu próprio Y_{ant} , objetos diferentes mantêm estados de alinhamento independentes, evitando interferências quando vários objetos percorrem curvas simultaneamente.

A matriz de rotação resultante é:

$$M_{align} = \begin{pmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Esta matriz é multiplicada à matriz de modelação corrente imediatamente após a translação para o ponto da curva, orientando o objeto ao longo da tangente.

3.3.2 Visualização da curva

Para permitir a verificação visual das trajetórias, as curvas de *Catmull-Rom* são desenhadas como uma linha fechada. A curva é mostrada em 100 pontos uniformemente distribuídos, calculados pela fórmula da posição acima, e desenhada com a primitiva `GL_LINE_LOOP` do OpenGL.

4 Bezier Patches

Foi adicionado ao *generator* suporte à geração de modelos a partir de ficheiros `.patch`, que descrevem *patches* de Bézier bicúbicos, grelha 4×4 de pontos de controlo, e gera malhas trianguladas para exportação no formato `.3d` já estabelecido nas fases anteriores.

4.1 Formato do ficheiro `.patch`

O ficheiro `.patch` tem a seguinte estrutura:

- Um inteiro com o número de *patches*, seguido dos 16 índices de cada *patch*, grelha 4×4 , varrida linha a linha, que referenciam os pontos de controlo.
- Um inteiro com o número total de pontos de controlo, seguido das coordenadas (x, y, z) de cada ponto.

4.2 Geração

Para cada *patch*, avalia-se a superfície para uma grelha de $(\text{tess} + 1) \times (\text{tess} + 1)$ pontos, onde *tess* é o nível de tesselação fornecido como parâmetro. Cada ponto $P(u, v)$ é calculado através da seguinte fórmula:

$$P(u, v) = \begin{pmatrix} u^3 & u^2 & u & 1 \end{pmatrix} \cdot M_B \cdot P_{ij} \cdot M_B^T \cdot \begin{pmatrix} v^3 & v^2 & v & 1 \end{pmatrix}^T$$

onde P_{ij} é a matriz 4×4 dos pontos de controlo do *patch* e M_B é a matriz de Bézier:

$$M_B = \begin{pmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Na prática, a avaliação é feita por componente (x, y, z) usando as bases de *Bernstein* de grau 3:

$$B_0(t) = (1 - t)^3, \quad B_1(t) = 3t(1 - t)^2, \quad B_2(t) = 3t^2(1 - t), \quad B_3(t) = t^3$$

$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B_j(v) P_{ij}$$

onde P_{ij} são os pontos de controlo do *patch*.

4.3 Normais

A normal em cada ponto é calculada como o produto vetorial das derivadas parciais da superfície em ordem a u e v :

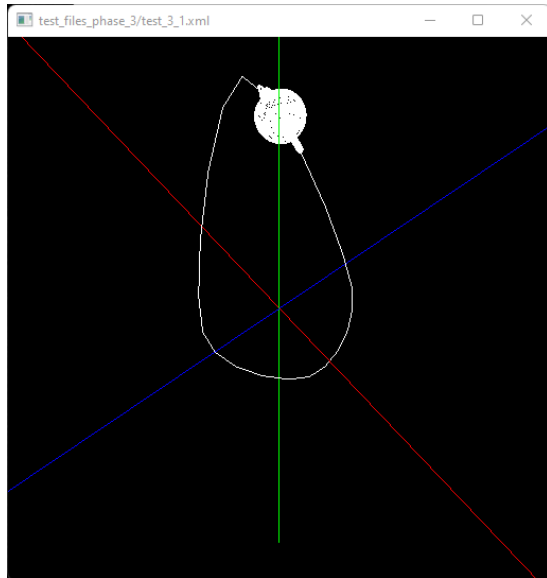
$$\frac{\partial P}{\partial u} = \sum_{i=0}^3 \sum_{j=0}^3 B'_i(u) B_j(v) P_{ij}, \quad \frac{\partial P}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B'_j(v) P_{ij}$$

$$n = \text{normalizar} \left(\frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v} \right)$$

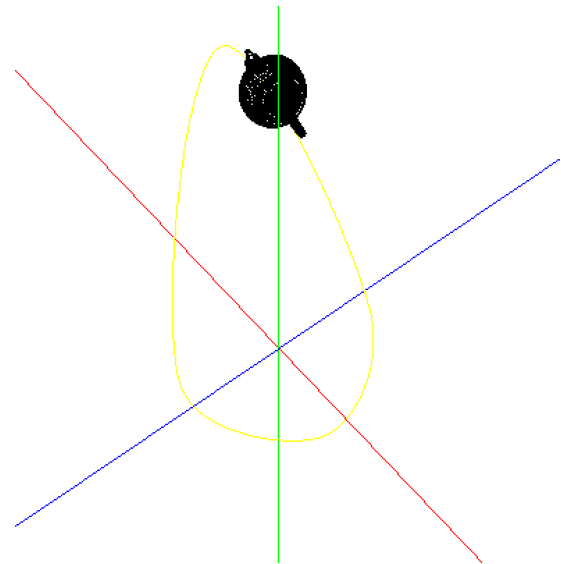
Cada *quad* da grelha de tesselação é subdividido em dois triângulos em ordem CCW.

5 Testes

Nesta secção são apresentados os resultados obtidos para os testes da Fase 3, comparados com os resultados esperados fornecidos pela equipa docente. Na pasta *reports* do repositório é ainda possível consultar a gravação do resultado do teste 1, que pode ser comparado com o resultado esperado fornecido.

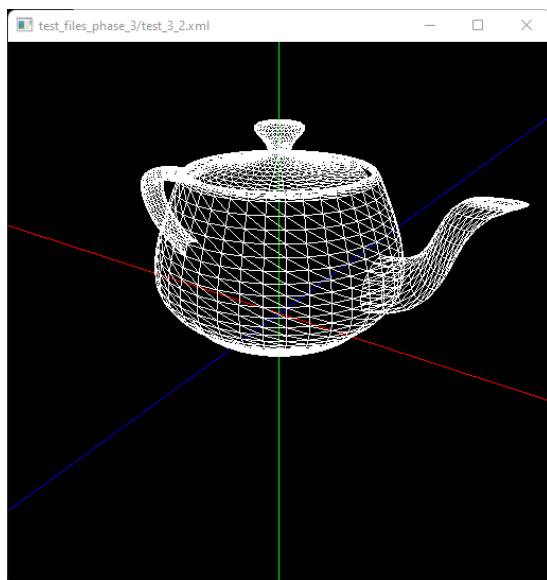


(a) Resultado Esperado

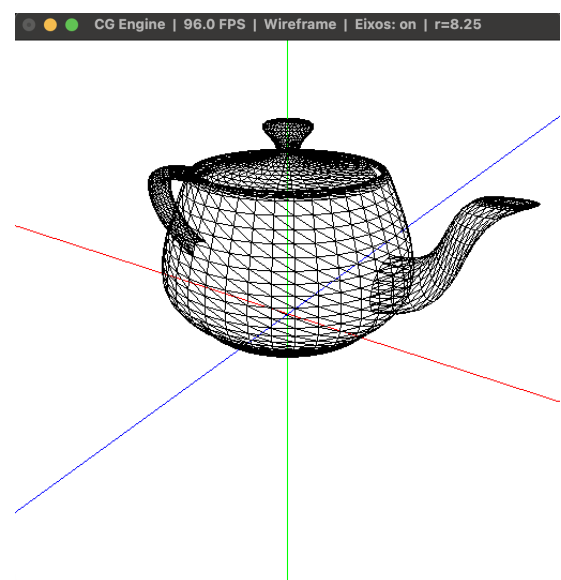


(b) Resultado Obtido

Figura 1: Teste 1



(a) Resultado Esperado



(b) Resultado Obtido

Figura 2: Teste 2

6 Sistema Solar Dinâmico

Na fase anterior, o sistema solar era estático. Nesta fase, a *scene* foi convertida num conjunto totalmente animado, com órbitas temporais para todos os corpos e com a introdução de um cometa cuja trajetória é definida por uma curva de *Catmull-Rom*. O modelo do cometa é gerado a partir de *patches* de *Bézier*, integrando assim as três funcionalidades principais desenvolvidas nesta fase. Na pasta *reports* do repositório é ainda possível consultar a gravação do resultado desta cena.

6.1 Planetas e luas

Cada planeta orbita o Sol com um período próprio, definido por uma rotação temporal em torno do eixo *Y*. A tabela seguinte lista os períodos definidos:

Corpo	Período orbital
Sol (auto-rotação)	25 s
Mercúrio	5 s
Vénus	10 s
Terra	15 s
Marte	25 s
Júpiter	40 s
Saturno	60 s
Urano	80 s
Neptuno	100 s

Tabela 1: Períodos orbitais dos planetas na *scene* de demonstração

As luas de cada planeta foram igualmente animadas com períodos orbitais próprios em torno do planeta-mãe. A hierarquia de grupos garante que as luas herdem a translação do planeta, orbitando corretamente enquanto o planeta se desloca em torno do Sol.

O anel de Saturno permanece estático relativamente ao planeta, pois a sua posição é definida por uma rotação estática de 25° em torno do eixo *X*, sem atributo *time*.

6.2 Cometa

Foi adicionado um cometa com trajetória elíptica inclinada, definida por 8 pontos de controlo numa curva de *Catmull-Rom* fechada. O cometa percorre a sua órbita em 35 segundos, com alinhamento ativo para que se oriente ao longo da trajetória em cada instante.

Os 8 pontos de controlo distribuem-se simetricamente em torno da origem a diferentes alturas, definindo uma elipse inclinada com semi-eixo maior de 30 unidades e variação em *Y* de ± 8 unidades. Esta inclinação dá ao cometa uma órbita claramente distinguível do plano eclíptico dos planetas.

O modelo do cometa é um *teapot* gerado a partir dos *patches* de *Bézier* fornecidos, com nível de tesselação 10, produzindo 6400 triângulos.

7 Conclusão

Nesta fase foram implementadas com sucesso as três componentes obrigatórias do projeto. No que respeita aos VBOs, a geometria dos modelos passou a ser carregada para a GPU uma única vez no arranque da aplicação, sendo a renderização feita diretamente a partir de *buffers* em GPU. Esta alteração elimina a dependência do modo imediato utilizado nas fases anteriores e melhora a eficiência da visualização.

Relativamente às transformações temporais, foram introduzidas rotações contínuas e translações ao longo de curvas de *Catmull-Rom*, com suporte para alinhamento automático do objeto à tangente da trajetória e para visualização da própria curva durante a execução. Por fim, no que diz respeito aos *Bezier Patches*, o gerador passou a produzir modelos a partir de ficheiros *.patch*, calculando as posições por bases de *Bernstein* e as normais através do produto vetorial das derivadas parciais da superfície.

A *scene* de demonstração, composta pelo sistema solar dinâmico com o cometa, integra estas funcionalidades e comprova o correcto funcionamento das animações hierárquicas, da geração de superfícies paramétricas e da respectiva renderização em tempo real.